

SummarizedExperiment

Kasper D. Hansen

- [Dependencies](#)
- [Corrections](#)
- [Overview](#)
- [Other Resources](#)
- [Start](#)
- [SessionInfo](#)

Dependencies

This document has the following dependencies:

```
library(GenomicRanges)
library(airway)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("GenomicRanges", "airway"))
```

Corrections

Improvements and corrections to this document can be submitted on its [GitHub](#) in its [repository](#).

Overview

We will present the SummarizedExperiment class from [GenomicRanges](#) package; an extension of the ExpressionSet class to include GRanges.

This class is suitable for storing processed data particularly from high-throughout sequencing assays.

Other Resources

Start

An example dataset, stored as a `SummarizedExperiment` is available in the [*airway*](#) package. This data represents an RNA sequencing experiment, but we will use it only for illustrating the class.

```
library(airway)
data(airway)
airway
```

```
## class: SummarizedExperiment
## dim: 64102 8
## exptData(1): ''
## assays(1): counts
## rownames(64102): ENSG000000000003 ENSG000000000005 ... LRG_98 LRG_99
## rowRanges metadata column names(0):
## colnames(8): SRR1039508 SRR1039509 ... SRR1039520 SRR1039521
## colData names(9): SampleName cell ... Sample BioSample
```

This looks similar to `ExpressionSet` and yet different from `ExpressionSet`. Things now have different names, representing that these classes were designed about 10 years apart.

We have 8 samples and 64102 features (genes).

Some aspects of the object are very similar to `ExpressionSet`, although with slightly different names and types:

`colData` contains phenotype (sample) information, like `pData` for `ExpressionSet`. It returns a `DataFrame` instead of a `data.frame`:

```
colData(airway)
```

```
## DataFrame with 8 rows and 9 columns
##           SampleName      cell      dex      albut      Run avgLength
##           <factor> <factor> <factor> <factor> <factor> <integer>
## SRR1039508 GSM1275862   N61311   untrt   untrt SRR1039508      126
## SRR1039509 GSM1275863   N61311    trt   untrt SRR1039509      126
## SRR1039512 GSM1275866  N052611   untrt   untrt SRR1039512      126
## SRR1039513 GSM1275867  N052611    trt   untrt SRR1039513       87
## SRR1039516 GSM1275870  N080611   untrt   untrt SRR1039516      120
## SRR1039517 GSM1275871  N080611    trt   untrt SRR1039517      126
## SRR1039520 GSM1275874  N061011   untrt   untrt SRR1039520      101
## SRR1039521 GSM1275875  N061011    trt   untrt SRR1039521       98
##           Experiment      Sample      BioSample
##           <factor> <factor> <factor>
## SRR1039508 SRX384345 SRS508568 SAMN02422669
## SRR1039509 SRX384346 SRS508567 SAMN02422675
## SRR1039512 SRX384349 SRS508571 SAMN02422678
## SRR1039513 SRX384350 SRS508572 SAMN02422670
## SRR1039516 SRX384353 SRS508575 SAMN02422682
## SRR1039517 SRX384354 SRS508576 SAMN02422673
## SRR1039520 SRX384357 SRS508579 SAMN02422683
## SRR1039521 SRX384358 SRS508580 SAMN02422677
```

You can still use \$ to get a particular column:

```
airway$cell
```

```
## [1] N61311 N61311 N052611 N052611 N080611 N080611 N061011 N061011
## Levels: N052611 N061011 N080611 N61311
```

exptData is like experimentData from ExpressionSet. This slot is often un-used; this is the case for this object:

```
exptData(airway)
```

```
## List of length 1
```

colnames are like sampleNames from ExpressionSet; rownames are like featureNames.

```
colnames(airway)
```

```
## [1] "SRR1039508" "SRR1039509" "SRR1039512" "SRR1039513" "SRR1039516"  
## [6] "SRR1039517" "SRR1039520" "SRR1039521"
```



```
head(rownames(airway))
```

```
## [1] "ENSG00000000003" "ENSG00000000005" "ENSG00000000419" "ENSG00000000460"  
## [5] "ENSG00000000005" "ENSG00000000009"
```



The measurement data are accessed by assay and assays. A SummarizedExperiment can contain multiple measurement matrices (all of the same dimension). You get all of them by assays and you select a particular one by assay(OBJECT, NAME) where you can see the names when you print the object or by using assayNames. In this case there is a single matrix called counts:

```
airway
```

```
## class: SummarizedExperiment  
## dim: 64102 8  
## exptData(1): ''  
## assays(1): counts  
## rownames(64102): ENSG00000000003 ENSG00000000005 ... LRG_98 LRG_99  
## rowRanges metadata column names(0):  
## colnames(8): SRR1039508 SRR1039509 ... SRR1039520 SRR1039521  
## colData names(9): SampleName cell ... Sample BioSample
```

```
assayNames(airway)
```

```
## [1] "counts"
```

```
assays(airway)
```

```
## List of length 1
## names(1): counts
```

```
head(assay(airway, "counts"))
```

```
##           SRR1039508 SRR1039509 SRR1039512 SRR1039513 SRR103951
## ENSG000000000003      679      448      873      408      113
## ENSG000000000005        0        0        0        0
## ENSG000000000419      467      515      621      365      58
## ENSG000000000457      260      211      263      164      24
## ENSG000000000460       60       55       40       35       7
## ENSG000000000938        0        0        2        0
##           SRR1039517 SRR1039520 SRR1039521
## ENSG000000000003     1047      770      572
## ENSG000000000005        0        0        0
## ENSG000000000419      799      417      508
## ENSG000000000457      331      233      229
## ENSG000000000460       63       76       60
## ENSG000000000938        0        0        0
```



So far, this is all information which could be stored in an ExpressionSet. The new thing is that SummarizedExperiment allows for a rowRanges (or granges) data representing the different features. The idea is that these GRanges tells us which part of the genome is summarized for a particular feature. Let us take a look

```
length(rowRanges(airway))
```

```
## [1] 64102
```

```
dim(airway)
```

```
## [1] 64102      8
```

```
rowRanges(airway)
```

```
## GRangesList object of length 64102:
## $ENSG000000000003
## GRanges object with 17 ranges and 2 metadata columns:
##      seqnames      ranges strand | exon_id exon_
##      <Rle>        <IRanges> <Rle> | <integer> <charac
## [1]      X [99883667, 99884983] - |    667145 ENSE0000145
## [2]      X [99885756, 99885863] - |    667146 ENSE0000086
## [3]      X [99887482, 99887565] - |    667147 ENSE0000040
## [4]      X [99887538, 99887565] - |    667148 ENSE0000184
## [5]      X [99888402, 99888536] - |    667149 ENSE0000355
## ...      ...      ...      ...
## [13]     X [99890555, 99890743] - |    667156 ENSE0000351
## [14]     X [99891188, 99891686] - |    667158 ENSE0000188
## [15]     X [99891605, 99891803] - |    667159 ENSE0000185
## [16]     X [99891790, 99892101] - |    667160 ENSE0000186
## [17]     X [99894942, 99894988] - |    667161 ENSE0000182
##
## ...
## <64101 more elements>
## -----
## seqinfo: 722 sequences (1 circular) from an unspecified genome
```

See how `rowRanges` is a `GRangesList` (it could also be a single `GRanges`). Each element of the list represents a feature and the `GRanges` of the feature tells us the coordinates of the exons in the gene (or transcript). Because these are genes, for each `GRanges`, all the ranges should have the same strand and `seqnames`, but that is not enforced.

In total we have around 64k “genes” or “transcripts” and around 745k different exons.

```
length(rowRanges(airway))
```

```
## [1] 64102
```

```
sum(elementLengths(rowRanges(airway)))
```

```
## [1] 745593
```

For some operations, you don’t need to use `rowRanges` first, you can use the operation directly on the object. Here is an example with `start()`:

```
start(rowRanges(airway))
```

```
## IntegerList of length 64102
## ["ENSG000000000003"] 99883667 99885756 99887482 ... 99891790 9989494
## ["ENSG000000000005"] 99839799 99840228 99848621 ... 99854013 9985450
## ["ENSG000000000419"] 49551404 49551404 49551433 ... 49574900 4957490
## ["ENSG000000000457"] 169818772 169821804 ... 169862929 169863148
## ["ENSG000000000460"] 169631245 169652610 ... 169821931 169821931
## ["ENSG000000000938"] 27938575 27938803 27938811 ... 27961576 2796157
## ["ENSG000000000971"] 196621008 196621186 ... 196716241 196716241
## ["ENSG000000001036"] 143815948 143817763 ... 143832548 143832548
## ["ENSG000000001084"] 53362139 53362139 53362940 ... 53429548 5348168
## ["ENSG000000001167"] 41040684 41040722 41046768 ... 41065096 4106509
## ...
## <64092 more elements>
```



```
start(airway)
```

```
## IntegerList of length 64102
## ["ENSG000000000003"] 99883667 99885756 99887482 ... 99891790 9989494
## ["ENSG000000000005"] 99839799 99840228 99848621 ... 99854013 9985450
## ["ENSG000000000419"] 49551404 49551404 49551433 ... 49574900 4957490
## ["ENSG000000000457"] 169818772 169821804 ... 169862929 169863148
## ["ENSG000000000460"] 169631245 169652610 ... 169821931 169821931
## ["ENSG000000000938"] 27938575 27938803 27938811 ... 27961576 2796157
## ["ENSG000000000971"] 196621008 196621186 ... 196716241 196716241
## ["ENSG000000001036"] 143815948 143817763 ... 143832548 143832548
## ["ENSG000000001084"] 53362139 53362139 53362940 ... 53429548 5348168
## ["ENSG000000001167"] 41040684 41040722 41046768 ... 41065096 4106509
## ...
## <64092 more elements>
```



You can use `granges` as synonymous for `rowRanges`.

Subsetting works like `ExpressionSet`: there are two dimensions, the first dimension is features (genes) and the second dimension is samples.

Because the `SummarizedExperiment` contains a `GRanges[List]` you can also use `subsetByOverlaps`, like

```
gr <- GRanges(seqnames = "1", ranges = IRanges(start = 1, end = 10^7))
subsetByOverlaps(airway, gr)
```

```
## class: SummarizedExperiment
## dim: 329 8
## exptData(1): ''
## assays(1): counts
## rownames(329): ENSG00000007923 ENSG00000008128 ... ENSG00000272512
##      ENSG00000273443
## rowRanges metadata column names(0):
## colnames(8): SRR1039508 SRR1039509 ... SRR1039520 SRR1039521
## colData names(9): SampleName cell ... Sample BioSample
```

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4      parallel  methods    stats      graphics  grDevices  utils
## [8] datasets    base
##
## other attached packages:
## [1] airway_0.102.0      GenomicRanges_1.20.5 GenomeInfoDb_1.4.2
## [4] IRanges_2.2.7       S4Vectors_0.6.3      BiocGenerics_0.14.0
## [7] BiocStyle_1.6.0      rmarkdown_0.7
##
## loaded via a namespace (and not attached):
## [1] digest_0.6.8      formatR_1.2        magrittr_1.5       evaluate_0.7.2
## [5] stringi_0.5-5     xVector_0.8.0      tools_3.2.1        stringr_1.0.0
## [9] yaml_2.1.13       htmltools_0.2.6    knitr_1.11
```

